

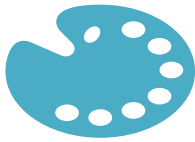


Mobile Devices and Applications

Unit 11 – 2D Graphics



Agenda



Two-dimensional Graphics

Images
Colors, tools, canvas area
Geometrical shapes



Capturing images using photo camera



Using images from the device gallery

Multimedia resources



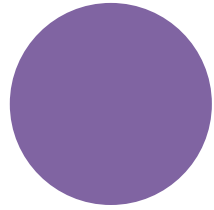
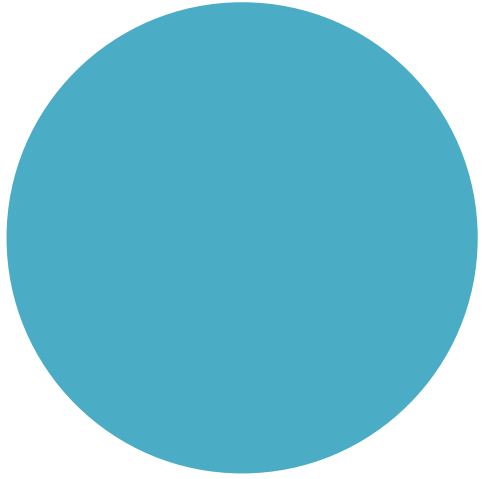
Files

Images
Audio files
Video files



Memory

2D or 3D graphics
Audio sequences
Animation



2D GRAPHICS



2D Graphics

- Drawing images in memory
 - Bitmap class
- Colors and textures
- The drawing tool
 - Paint class
- Drawing surfaces
 - Canvas class
- Geometrical figures
 - ShapeDrawable

Displaying images inside controls

- ImageView
- Drawable (abstract class)
 - res/drawable, res/raw
 - Images
 - Image files
 - Shapes
 - XML files
- Canvas
 - android.graphics
 - Free drawings

Images

- **Bitmap**
- Editable or non editable
- Static methods
 - **createBitmap**(width, height, config)
 - Bitmap.Config
 - **createBitmap**(srcBitmap) and other overloads
 - **createScaledBitmap**(src, wDest, hDest, filter)
- Methods
 - **compress**(format, quality, output_stream)
 - **getWidth()**, **getHeight()**
- **BitmapFactory**
 - Creating Bitmap objects from different sources
 - Static methods ***decode***SOURCE()
 - Resource, Stream, File etc.

Images

```
int width, height;
```

```
Bitmap bmp1 = Bitmap.createBitmap(width, height,  
Bitmap.Config.ARGB_8888);
```

```
Bitmap bmp2 = BitmapFactory.decodeResource(  
    getResources(), R.drawable.dice);
```

Basic components

- Colours and textures
- Drawing tool
- Drawing surfaces
- Shape objects

Colors

- **Color** class
 - Constants (**RED**, **BLUE** etc.)
 - Static method **argb**(a, r, g, b)
 - Static method **parseColor**(String)
 - #RRGGBB
 - #AARRGGBB
 - Color name
- Resources
 - **Resources#getColor()**
 - res/values

Colors



```
int color =  
    getResources().getColor(R.id.fundal);
```



```
int green = Color.GREEN;
```



```
int semiTransparentGreen =  
    Color.argb(127, 0, 255, 0);
```

Gradients

- Used for colors and images
 - Shader class and its derived forms
- Filling mechanisms:
 - Repeating the template (**Shader.TileMode.REPEAT**),
 - Repeating the template and mirroring the image (**Shader.TileMode.MIRROR**)
 - Reusing the external color when boundaries are being trespassed (**Shader.TileMode.CLAMP**).

Gradients

- **BitmapShader** – drawing images
- **LinearGradient** – linear gradients
- **RadialGradient** – circular gradients
- **SweepGradient** – angled gradients
- **ComposeShader** – combining two gradients

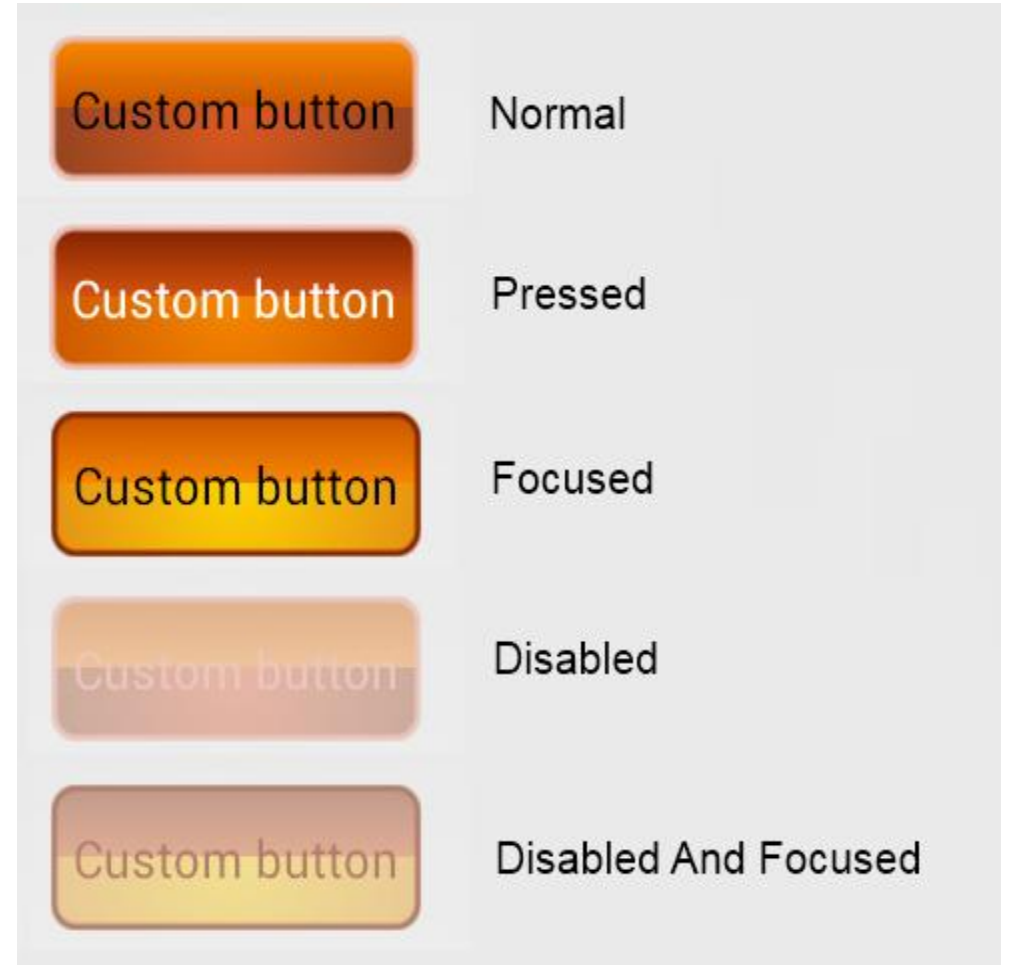
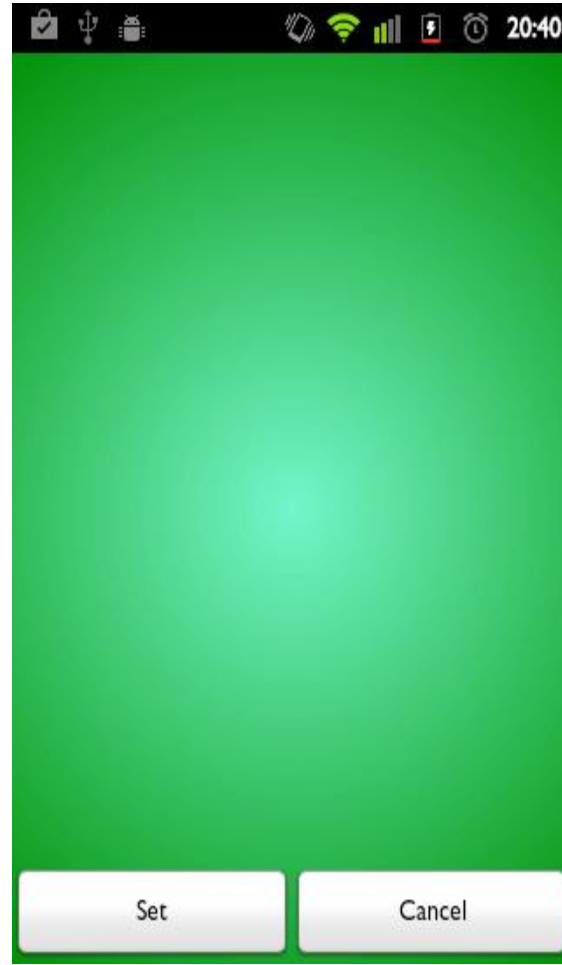
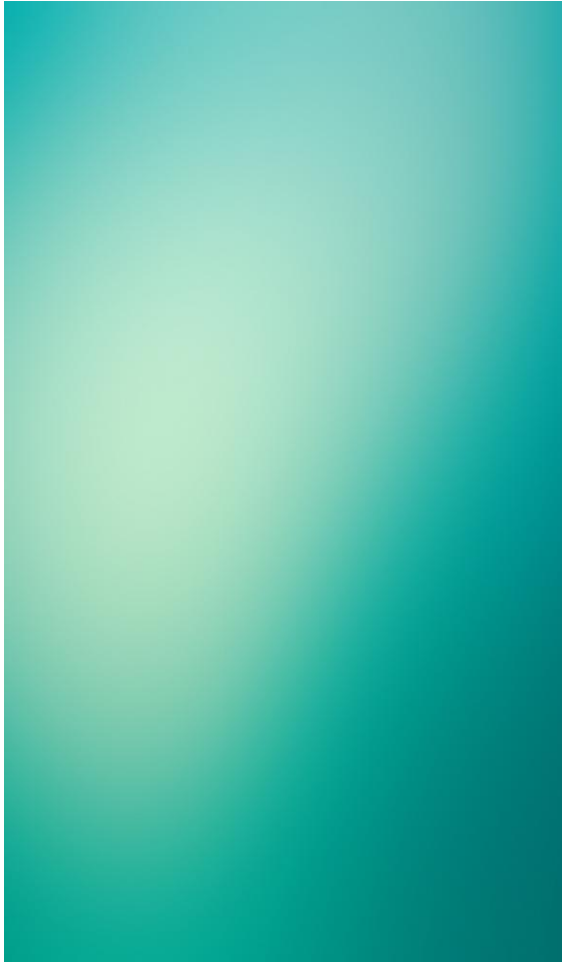
Gradients

```
Bitmap bmpShader;
```

```
BitmapShader gradientBmp = new  
    BitmapShader(bmpShader,          Shader.TileMode.REPEAT,  
    Shader.TileMode.REPEAT );
```

```
LinearGradient gradientLinear = new LinearGradient(0, 0, 150,  
150, Color.RED, Color.BLUE, Shader.TileMode.CLAMP);
```

Gradients



Drawing instrument – Paint class

- Controls the properties of the drawn objects like shapes, text and images
- Includes methods like
 - color
 - width
 - line style
 - font
 - other effects
- Most of the drawing methods have a parameter of type Paint

Paint class settings

Color

```
void setColor(int color)  
void setARGB(int a, int r, int  
g, int b)
```

Text

```
void setTextAlign(Paint.Align  
align)  
void setTextSize(float  
textSize)
```

Paint class

- Other properties:
 - void **setStyle**(Paint.Style style)
 - void **setStrokeWidth**(float width)
 - PathEffect **setPathEffect**(PathEffect effect)
 - void **setAntialias**(boolean aa)
- Initializing gradients
 - Shader **setShader**(Shader shader)

Paint class

```
Paint creionGrad = new Paint(), creionNegru = new Paint(), creionText = new Paint();
```

```
creionNegru.setColor(Color.BLACK);  
creionNegru.setStyle(Paint.Style.STROKE);  
creionNegru.setAntiAlias(true);
```

```
creionText.setTextAlign(Paint.Align.CENTER);  
creionText.setColor(Color.GREEN);  
creionText.setTextSize(24);
```

```
creionGrad.setShader(gradientBmps);  
creionGrad.setStrokeWidth(30);
```

Drawing surface – Canvas class

- Methods for drawing shape, text and images
- draw...()
- Needs a drawing instrument
- Position
- Additional effects

Drawing inside controls

- The class must extent the View class
- The main method used is **onDraw()**
- The method is called whenever the objects needs to be drawn
- Other important methods
 - **invalidate()** – from the main UI thread;
 - **postInvalidate()** – from a background thread;

Canvas

```
public class GraphView extends View {  
    ...  
    protected void onDraw(Canvas canvas) {  
        super.onDraw(canvas);  
        //drawing objects  
    }  
}
```

Drawing bitmaps

- Constructor
 - **Canvas(Bitmap)**
- Steps
 - Creating or getting a bitmap object
 - **Bitmap bmp = Bitmap.createBitmap(l, L, tip);**
 - Initializing the canvas object
 - **Canvas canvas = new Canvas(bmp);**

Drawing geometric figures

- void **drawPoint**(float x, float y, Paint paint)
- void **drawPoints**(float[] pts, int offset, int count, Paint paint)
- void **drawLine**(float startX, float startY, float stopX, float stopY, Paint paint)
- void **drawLines**(float[] pts, Paint paint)

Drawing geometric figures

- void **drawRect**(RectF rect, Paint paint)
- void **drawRoundRect**(RectF rect, float rx, float ry, Paint paint)
- void **drawCircle**(float cx, float cy, float radius, Paint paint)
- void **drawOval**(RectF oval, Paint paint)
- void **drawArc**(RectF oval, float startAngle, float sweepAngle, boolean useCenter, Paint paint)

Displaying images

- `void drawBitmap(Bitmap bmp, float left, float top, Paint paint)`
 - `Bitmap bmp = BitmapFactory.decodeResource(getResources(), R.drawable.id);`
- `void drawPicture(Picture picture, RectF dst)`

Changing the background

- void **drawColor**(int color)
- void **drawRGB**(int r, int g, int b)
- void **drawARGB**(int a, int r, int g, int b)
- void **drawPaint**(Paint paint)

Drawing text

- void **drawText**(String text, float x, float y, Paint paint)

Drawing shapes

- Support for vector graphics
- Drawing geometrical shapes:
 - **addCircle()**
 - **addRect()**
 - **addArc()**
 - etc.
- Combining shapes
 - **addPath()**

Example: Canvas

```
//background color
canvas.drawColor(Color.YELLOW);
//line
canvas.drawLine(10, 10, 300, 4000, pencil);
//circle
canvas.drawCircle(100, 100, 50, pencil);
// rectangle
canvas.drawRect(rect, pencil); //-----rect of type Rect or RectF
//ellipse
canvas.drawOval(rect, pencil);
// Add a specified arc to the rect as a new contour
path.addArc(rect, 270, 90);
// draw a path
canvas.drawPath(path, pencil);
// draw a text over a path
canvas.drawTextOnPath("Arc contour", path, 0, 40, pencil);
```

Drawing a text over a path

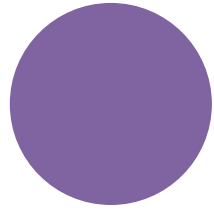
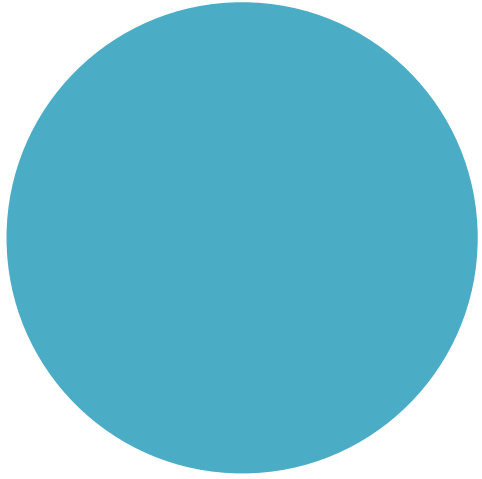
- void **drawPath**(Path path, Paint paint)
- void **drawTextOnPath**(String text, Path path, float hOffset, float vOffset, Paint paint)

Cutting

- Clipping area - clip
 - Sets the area which will be modified by using the Canvas draw method
 - Initially the clip is set onto the entire window
- Modifying the current clip:
 - **clipRect()**
 - **clipPath()**
 - **clipRegion()**

Transformations

- Scaling
 - **scale**(sx, sy)
- Translates (changing coordinates)
 - **translate**(dx, dy)
- Rotations
 - **rotate**(grade)
- Angle shifting
 - **skew**(x, y)



GEOMETRICAL FIGURES



GEOMETRICAL FIGURES

- ShapeDrawable
- Shape objects
 - RectShape
 - OvalShape
 - ArcShape
 - PathShape
- Methods
 - getPaint()
 - draw(Canvas)
 - setBounds(x1, y1, x2, y2)

GEOMETRICAL FIGURES

ShapeDrawable drawable;

Canvas canvas;

int x, y, width, height;

drawable = new ShapeDrawable(new RectShape());

drawable.getPaint().setColor(Color.RED);

drawable.setBounds(x, y, x + width, y + height);

drawable.draw(canvas);

Shapes (XML)

- **Shape** element
- Shape attribute **android:shape**
 - **rectangle**
 - **oval**
 - **line**
 - **ring**
- Properties
 - **stroke**
 - **solid**
 - **gradient**
 - **size**

Shapes (XML)

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle" >
    <corners android:radius="10dp" />
    <size
        android:height="@dimen/lat"
        android:width="@dimen/lung" />
    <solid
        android:color="@android:color/transparent" />
    <stroke
        android:width="2dp"
        android:color="#000000" />
</shape>
```

Shapes (XML)

- XML reference :
 - @drawable/fileNameForShapes
- Java reference:
 - R.drawable. fileNameForShapes
 - Resources#getDrawable

Animation

```
ImageView imageView = (ImageView) findViewById(R.id.animationView);  
imageView.setBackgroundResource(R.drawable.swing_animation);  
  
// Get the background, which has been compiled to an  
// AnimationDrawable object.  
AnimationDrawable frameAnimation = (AnimationDrawable)  
imageView.getBackground();  
  
// Start the animation (looped playback by default).  
frameAnimation.start();
```

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>